Conditionals: if, elif, else

Comparisons that give True or False, and the if, elif and else lines that choose which block of code runs.

THE QUESTION

How can a program do different things for different inputs?

THE INTUITION

A comparison such as `x > 10` gives `True` or `False`. An `if` runs its indented block only when its test is True; `elif` and `else` add more paths, and Python takes the first path whose test is True.

KEY TAKEAWAY

In an if/elif/else chain exactly one block runs: the first whose test is True, else the `else` block. Indentation decides which lines belong to it.

a value: x = 9 → a test: x > 10 → True or False → if picks a block → the lines after the chain run either way

1 Comparisons answer with True or False

expression	asks: is x ...	x = 9	x = 10	x = 12
`x > 10`	greater than 10?	False	False	True
`x < 10`	less than 10?	True	False	False
`x == 10`	equal to 10?	False	True	False
`x != 10`	not equal to 10?	True	False	True
`x >= 10`	greater than or equal to 10?	False	True	True
`x <= 10`	less than or equal to 10?	True	True	False

`print(type(x > 10))` shows `<class 'bool'>` True, False: no quotes

A **comparison operator** compares two values and gives a **Boolean**, of type `bool`: `True` or `False`. == (two signs) asks "equal?", `!=` asks "not equal?". Numbers compare by value: `3 == 3.0` is `True`. Text must match exactly: `"cat" == "Cat"` is `False`. A comparison that decides what runs is a **condition**.

2 `if` runs a block only when its test is True

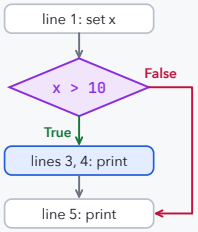
test-conditional.py

```
1 x = 9
2 if x > 10:
3     print("Greater than 10")
4     print("Yay!")
5 print("Program complete")
```

	x = 9	x = 12
	False	True
		True
		True
	True	True

x = 9 `Program complete`

x = 12 `Greater than 10` `Yay!` `Program complete`



`if`, a condition and a colon make the **header**. The indented lines under it are its **block**. True: the block runs. False: Python skips it. Either way it goes on to line 5. Each column is one run (line 1 set to 9, then 12): True/False is the test's result, a filled dot a line that ran, a hollow dot a line skipped.

3 `else`: exactly one of two blocks runs

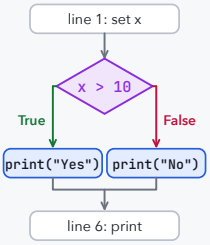
if / else

```
1 x = 9
2 if x > 10:
3     print("Yes")
4 else:
5     print("No")
6 print("We are done with checks")
```

	x = 9	x = 12
	False	True
		True
	True	
	True	True

x = 9 `No` `We are done with checks`

x = 12 `Yes` `We are done with checks`



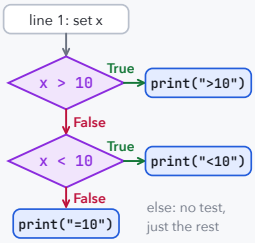
`else`: has no test; its block runs when the `if` test is False. So exactly one of `Yes` and `No` prints: a **two-way branch**. A **flowchart** draws it as a diamond (the question) with two arrows out. The course's `twoway.py` has the same shape.

4 `elif` adds tests; the first True one wins

threeway.py

```
1 x = 9
2 if x > 10:
3     print(">10")
4 elif x < 10:
5     print("<10")
6 else:
7     print("=10")
```

	x = 9	x = 10	x = 11
	False	False	True
			True
	True	False	
prints	<10	=10	>10



`elif` ("else if") is another test, checked only when every test above it was False. Python runs the first block whose test is True, then jumps past the whole chain: for `x = 11` line 4 is never checked. `else` takes what is left. A chain has one `if`, any number of `elif`s and at most one `else`, last.

5 Indentation decides what is inside the block

line 5 at the left edge

```
1 x = 9
2 if x > 10:
3     print("Greater than 10")
4     print("Yay!")
5 print("Program complete")
```

x = 9 prints `Program complete`

line 5 indented 4 spaces

```
1 x = 9
2 if x > 10:
3     print("Greater than 10")
4     print("Yay!")
5     print("Program complete")
```

x = 9 prints *(nothing)*

```
1 if x > 10:
2 print("Greater than 10")
```

no indent under the if: Python refuses to start

```
IndentationError: expected an indented block after 'if' statement on line 1
```

Python has no brackets around a block: **indentation**, the spaces at the start of a line, marks it. Use 4 spaces on every line of the block. Indent line 5 and it joins the block, so for `x = 9` nothing prints. A header with nothing indented under it is an **IndentationError**, reported before any line runs.

6 `and`, `or` and `not` combine conditions

a	b	a and b	a or b	not a
False	False	False	False	True
False	True	False	True	True
True	False	False	True	False
True	True	True	True	False

and: True only when both sides are True.
or: True when at least one side is True. not: flips True and False.

Logical operators combine Booleans; the table lists every case. With `has_ticket = True` and `has_id = False`, `has_ticket and has_id` is `False` (no entry) and `has_ticket or has_id` is `True`. Python may skip the right side once the answer is known: that is in the next sheet.

Combine tests, then put them in the right order

Range checks, which operator goes first, why the order of elif branches changes the answer, and = versus ==.

7 A range check needs two comparisons

`0 <= m <= 100` means the same as `m >= 0 and m <= 100`



(positions not to scale; each result printed by both forms)

Order when there are no brackets

`1 + - * / %` `2 < > == != <= >=` `3 not` `4 and` `5 or`

```
print(True or False and False)        True   and runs first
print((True or False) and False)      False  brackets run first
```

"Is `m` from 0 to 100?" is two questions joined by `and`. Python also lets you **chain** them, `0 <= m <= 100`, and reads that the same way. Without brackets Python applies arithmetic first, then comparisons, then `not`, `and`, `or`. So in `a and b == c` the `==` is worked out first.

8 The order of `elif`s changes the grade

wrong order

```
1 if marks >= 50:
2     grade = "C"
3 elif marks >= 75:
4     grade = "B"
5 elif marks >= 90:
6     grade = "A"
7 else:
8     grade = "F"
```

right order

```
1 if marks >= 90:
2     grade = "A"
3 elif marks >= 75:
4     grade = "B"
5 elif marks >= 50:
6     grade = "C"
7 else:
8     grade = "F"
```

rose lines: never run for any marks 0 to 100 (all 101 tried)

marks	wrong	right
95	C	A
80	C	B
60	C	C
30	F	F

marks = ... then the chain, then print(grade)

Same four tests, different order. The wrong chain checks `>= 50` first: 95 and 80 pass it, get C, and the chain stops. Put the strictest test first. Separate `if`s fail too: every True test runs, so 95 ends with `C`.

9 Even or odd: test the remainder

```
1 n = 7
2 if n % 2 == 0:
3     print(n, "is even")
4 else:
5     print(n, "is odd")
```

n	n % 2	n % 2 == 0	output
7	1	False	7 is odd
10	0	True	10 is even
0	0	True	0 is even
-3	1	False	-3 is odd

line 1 set to each n in turn; the remainder of -3 by 2 is 1 in Python

`n % 2` is the remainder after dividing by 2 (see the variables sheet): 0 for even numbers, 1 for odd ones. Test one input per branch plus the edges: 0 is even, and negative numbers work too.

10 = stores a value; == compares two

REPL

```
>>> x = 10
>>> x == 10
True
>>> x == 7
False
>>> x
10
```

`x = 10`
assignment: attach the name x to 10. A statement; shows nothing.

`x == 10`
comparison: is x equal to 10? An expression: True or False.

`if x = 10:` in a script stops before running with

```
SyntaxError: invalid syntax. Maybe you meant '==' or ':=' instead of '='?
```

One = is an **assignment**: it stores a value and shows nothing, even in the REPL. == is a comparison: it gives True or False and changes nothing. An `if` needs a condition, so `if x = 10:` is a SyntaxError.

11 Common mistakes: what you wrote, what Python says, the fix

you wrote	the last line Python prints; why, and the fix
`if x = 10:`	`SyntaxError: invalid syntax. Maybe you meant '==' or ':=' instead of '='?` one = stores; compare with ==
`if x > 5`	`SyntaxError: expected ':'` every header ends with a colon
`if x > 5:` `print("big")`	`IndentationError: expected an indented block after 'if' statement on line 1` indent the block by 4 spaces
`else: print("small")` `elif x == 5: print("five")`	`SyntaxError: invalid syntax` `elif` must come before `else`
`age = "18"` `if age >= 18:`	`TypeError: '>=' not supported between instances of 'str' and 'int'` text cannot be ordered against a number: `int(age) >= 18`
`x = 5` `if x == 1 or 2:`	`prints: x is 1 or 2` Python reads `(x == 1) or 2`, which counts as true; write `x == 1 or x == 2`

12 Try it (answers in green)

1. `x = 7`, then `print(x > 5 and x % 2 == 0)`?
`False`: 7 is odd.

2. `print(True and False == False)`?
`True`: == goes first and gives True.

3. `age < 12` pays 50, `elif age >= 60` 70, else 100. Ages 12, 60, 59?
100, 70, 100: 12 is not under 12.

4. A test that is True when y is outside 1 to 10?
`y < 1 or y > 10` (0, 5, 11 give True, False, True).

5. `t = 25`; `if t > 20:` print warm; then `if t > 10:` print mild, `else:` cold?
warm, mild: two separate ifs.

6. `if a:` print 1, `elif b:` print 2. Which a, b print nothing?
Only a and b both False.

a test gives True or False · the first True branch runs · the indented lines are the block · == compares, = stores

Source: the strings and conditionals notebook (the Booleans and if/else cells), the handwritten conditionals notes (code beside its flowchart), and the scripts twoway.py, threeway.py and test-conditional.py, nipunbatra.github.io/comp22. Every output, error message and ran/skipped mark was produced by running the code with Python 3.11.11. Comparing decimals such as 0.1 + 0.2 with ==, and and/or stopping early, come next. Next: the floating point and short-circuit sheet.